

Description d'un réseau de transport *UrbanLoop* à partir d'un fichier *JSON*

Tristan Le Godais
Malo Mongard

Encadrant : Thibault Cholez

urbanLOOP



Table des matières

1	Introduction	3
2	Objet "network"	3
3	Objet "view_box"	4
4	Objet "loop"	5
5	Objet "bridge"	6
6	Objets "element"	6
6.1	Gare : objet "station"	7
6.2	Dépôt : objet "shed"	7
6.3	Capteur : objet "sensor"	8
6.4	Aiguillage : objet "switch_in" ou "switch_out"	8
7	Objets "section"	9
8	Objets "pods"	10
9	Objet "pod"	10
10	Objet "travelers"	11
10.1	Pour les gares	11
10.2	Pour les capsules	12
11	Objets "place"	12
12	Exemple de réseau complet	12

1 Introduction

Ce document a pour but de spécifier la forme des fichiers *JSON* utilisés par le simulateur de réseaux *UrbanLoop*. Ces fichiers permettent de construire des réseaux. Si un éditeur est fait dans la suite, cette documentation peut être un outil de base pour bien comprendre comment spécifier un réseau.

Remarque : ne pas confondre le modèle et le fichier *JSON* ! Ce dernier sert à la fois à la construction du modèle, et à la vue.

La description du fichier va se faire par objet par objet, en commençant par l'objet "network" qui se situe à la racine du fichier.

2 Objet "network"

Le fichier *JSON* d'un réseau présente les attributs suivants :

"running" (booléen)

un booléen indiquant si la simulation est déjà lancée (car on peut faire une sauvegarde d'un réseau en cours de simulation)

"rate" (entier positif)

le logarithme en base 2 de la vitesse de la simulation

"max_rate" (entier positif)

le logarithme en base 2 de la vitesse maximale de la simulation

"jerky" (booléen)

un booléen indiquant l'algorithme utilisé pour accélérer la simulation

"time" (flottant positif)

le temps simulé de reprise de la simulation (en s)

"state" (flottant)

un *seed* permettant de choisir le générateur pseudo-aléatoire utilisé et donc de relancer une même simulation dans les mêmes conditions

"name" (chaîne de caractères)

le nom du réseau

"view_box" (objet "view_box")

une *view box* contenant les dimensions du réseau pour la vue

"loops" (liste d'objets "loop")

une liste de boucles

"bridges" (liste d'objets "bridge")

une liste de ponts

"margin_min" (flottant positif)

la marge minimale entre deux capsules (la marge sur les boucles ayant la plus petite vitesse)

"max_speed" (flottant positif)

la vitesse maximale autorisée pour les capsules (en m / s)

"pod_size" (flottant positif)

la taille d'une capsule (en m)

"places_number" (entier positif)

le nombre de places du rouage de l'algorithme d'aiguillage

"dynamic_routing" (booléen)

une option pour activer ou non la mise à jour des tables de routage

Voici un exemple (les objets "loop", "bridge" et "view_box" seront présentés plus tard dans ce document) :

```
1  Représentation d'un objet "network" :
2  {
3      "running": false ,
4      "rate:" 0,
5      "max_rate": 7,
6      "jerky": false ,
7      "time": 28800,
8      "state": 42,
9      "name": "GrandNancy" ,
10     "view_box": {...} ,
11     "loops": [...],
12     "bridges": [...],
13     "margin_min": 2,
14     "max_speed": 30,
15     "pod_size": 2,
16     "places_number": 10,
17     "dynamic_routing": false
18 }
```

3 Objet "view_box"

Une *view box* sert au dimensionnement du réseau dans la vue et est spécifiée avec les attributs suivants :

"x" (flottant)

une abscisse

"y" (flottant)

une ordonnée

"width" (flottant positif)

une largeur

"height" (flottant positif)

une hauteur

En voici un exemple :

```
1  Représentation d'une view box :
2  {
3      "x": 0,
4      "y": 0,
5      "width": 1200,
6      "height": 850
7  }
```

4 Objet "loop"

Les boucles sont spécifiées avec les attributs suivants :

"name" (chaîne de caractères)

le nom de la boucle

"elements" (liste d'objets "elements")

une liste ordonnée d'éléments, qui sont de cinq types (ils seront détaillés dans la suite) :

- aiguillage entrant (objet "switch_in")
- aiguillage sortant (objet "switch_out")
- gare (objet "station")
- dépôt (objet "shed")
- capteur (objet "sensor")

Cette liste commence OBLIGATOIREMENT par un aiguillage entrant et les aiguillages doivent apparaître PAR PAIRE dans le réseau (à un entrant correspond aussi un sortant).

"sections" (liste d'objets "section")

une liste de sections de route

"pods" (liste d'objets "pod")

une liste de capsules en mouvement sur les sections

En voici un exemple :

```
1  Représentation d'un objet "loop" :
2  {
3      "name": "Nancy",
4      "elements": [...],
5      "sections": [...],
6      "pods": {...}
7  }
```

5 Objet "bridge"

Les ponts sont spécifiés avec les attributs suivants :

"name" (chaîne de caractères)

le nom du pont

"section" (objet "section")

une section qui correspond à l'unique section de route du pont

"pods" (liste d'objets "pod")

une liste de capsules

"switch_in" (objet "place")

l'aiguillage entrant lié au pont

"switch_out" (objet "place")

l'aiguillage sortant lié au pont

En voici un exemple :

```
1  Représentation d'un objet "bridge" :  
2  {  
3      "name": "Pont Nancy-Laxou",  
4      "section": {...},  
5      "pods": {...},  
6      "switch_in": {...},  
7      "switch_out": {...}  
8  }
```

6 Objets "element"

Voici la représentation des cinq sortes d'objets qui peuvent être spécifiés dans les boucles. Il ne faut pas oublier de commencer la liste d'éléments par un aiguillage entrant (un objet "switch_in"). Tous les éléments ont en commun les attributs suivants :

"name" (chaîne de caractères)

le nom de l'élément

"type" (chaîne de caractères)

un type d'élément

"x" (flottant)

une abscisse

"y" (flottant)

une ordonnée

"pods" (objet "pods" ou liste d'objets "pods")

un objet modélisant les capsules au sein de l'élément (sauf pour le capteur)

Les champs spécifiés dans la suite sont les champs supplémentaires spécifiques aux éléments.

6.1 Gare : objet "station"

Une gare est spécifiée avec les attributs suivants :

"travelers" (objet "travelers")

un objet modélisant les voyageurs en train d'attendre une capsule dans la gare, ou y ayant attendu

"station_type" (entier entre 0 et 2)

un type de station qui influe sur la génération des voyageurs, parmi les trois suivant :

- zone d'activité : 0
- zone résidentielle : 1
- ville : 2

En voici un exemple :

```
1  Représentation d'une gare :
2  {
3      "name": "Stanislas",
4      "type": "station",
5      "pods": {...},
6      "travelers": {...},
7      "station_type": 2,
8      "x": 450,
9      "y": 320
10 }
```

6.2 Dépôt : objet "shed"

Un dépôt n'a pas d'attribut supplémentaire ; en voici un exemple :

```
1  Représentation d'un dépôt :
2  {
3      "name": "Dépôt Est",
4      "type": "shed",
5      "pods": {...},
6      "x": 560,
```

```
7     "y": 425
8 }
```

6.3 Capteur : objet "sensor"

Un capteur n'a pas d'attribut supplémentaire, ni même d'attribut "pods"; en voici un exemple :

```
1  Représentation d'un capteur :
2  {
3      "name": "Capteur Nancy Nord",
4      "type": "sensor",
5      "x": 560,
6      "y": 425
7  }
```

6.4 Aiguillage : objet "switch_in" ou "switch_out"

Il y a deux types d'aiguillage (entrant et sortant), mais ils sont modélisés de la même manière; seul l'attribut "type" diffère. Il ne faut surtout pas oublier de commencer la liste des éléments d'une boucle par un aiguillage entrant (un objet "switch_in")! Un aiguillage est spécifié avec les attributs suivants :

"id_bridge" (entier positif)

un entier qui renseigne l'indice du pont auquel est lié l'aiguillage

Voici un exemple pour chacun des deux types d'aiguillage :

```
1  Représentation d'un objet "switch_in" :
2  {
3      "name": "Aiguillage Nancy←Laxou",
4      "type": "switch_in",
5      "pods": [...],
6      "x": 600,
7      "y": 550,
8      "id_bridge": 0
9  }
```

```
1  Représentation d'un objet "switch_out" :
2  {
3      "name": "Aiguillage Nancy→Laxou",
4      "type": "switch_out",
```



```

5     "pods": [...],
6     "x": 560,
7     "y": 490,
8     "id_bridge": 0
9 }

```

Lors de la sérialisation d'un aiguillage les informations suivantes sont ajoutées :

"length" (flottant positif)

la longueur de l'aiguillage (en m)

"cursor" (flottant positif)

la position du curseur, pour les aiguillages entrant seulement

"discrete_places" (listes d'entiers positifs)

la liste des indices des capsules discrétisées (pour les aiguillages entrant)

7 Objets "section"

Les sections sont spécifiées avec les attributs suivants :

"name" (chaîne de caractères)

le nom de la section

"speed" (flottant positif)

une vitesse (en m / s)

"path" (objet "path")

un chemin donnant la forme de la section (dont les coordonnées des points de passage ou des points d'inflexion sont en m), dont le format compte s'inspirer de la grammaire de l'attribut "d" en SVG (<https://svgwg.org/svg2-draft/paths.html#PathDataBNF>), mais reste à définir précisément

En voici un exemple :

```

1  Représentation d'un objet "section" :
2  {
3      "name": "Avenue du Général Leclerc (Nancy)",
4      "speed": 22.22,
5      "path": {
6          "type": "line"
7      }
8  }

```

8 Objets "pods"

Pour les gares et les dépôts, les capsules sont spécifiées avec les attributs suivants :

"count" (entier positif)

le nombre de capsules actuellement stockées

"max" (entier positif)

le nombre de capsules maximal au sein de la gare ou du dépôt

En voici un exemple :

```
1 Représentation d'un objet "pods" :  
2 {  
3   "name": "Super Capsule",  
4   "count": 1,  
5   "max": 4  
6 }
```

Remarque : *a priori* lors de la spécification d'un réseau avant son démarrage on a pour les dépôts $count = max$ et pour les gares $count = 0$.

9 Objet "pod"

Pour les sections et les aiguillages, une capsule est spécifié avec les attributs suivants :

"name" (chaîne de caractères)

le nom de la capsule

"travelers" (objet "travelers")

un nombre de voyageurs

"source" (objet "place")

une gare ou un dépôt de départ

"destination" (objet "place")

une gare ou un dépôt d'arrivée

"position" (flottant positif)

une position relative au début de la boucle, c'est-à-dire la distance depuis le début de la boucle, qui est défini comme son premier aiguillage entrant (en m)

"speed" (flottant positif)

une vitesse (en m / s)

Les attributs suivants sont générés lors de la sérialisation si une capsule était dans un aiguillage ; il est déconseillé d'essayer de prévoir des simulations où des capsules seraient déjà dans des aiguillages car cela nécessite de calculer toutes ces informations :

"speed_restore" (flottant positif)

la vitesse de restauration (en m / s)

"length_before_restore" (flottant positif)

la longueur avant restauration de la vitesse (en m)

"length_before_turn" (flottant positif)

la longueur avant de tourner (en s)

En voici un exemple :

```
1  Représentation d'un objet "pod" :
2  {
3      "travelers": {...},
4      "source": {...},
5      "destination": {...},
6      "position": 237,
7      "speed": 22.22
8  }
```

10 Objet "travelers"

10.1 Pour les gares

Pour les gares, les voyageurs sont spécifiés avec les attributs suivants :

"count" (entier positif)

le nombre de voyageurs en attente de monter dans une capsule

"all_time_count" (entier positif)

le nombre total de voyageurs ayant attendus dans la gare depuis le début

"average_waiting_time" (flottant positif)

le temps moyen d'attente d'un voyageur pour monter dans une capsule au sein de la gare en seconde (en s)

En voici un exemple :

```
1  Représentation d'un objet "travelers" pour les gares :
2  {
3      "count": 6,
4      "all_time_count": 189,
5      "average_waiting_time": 120
6  }
```

10.2 Pour les capsules

Pour les capsules, les voyageurs sont spécifiés de la même façon que l'objet "pods" des gares et dépôts :

"count" (entier positif)

le nombre de voyageurs dans la capsule

"max" (entier positif)

le nombre maximum de voyageurs dans la capsule

En voici un exemple :

```
1  Représentation d'un objet "travelers" pour les capsules :
2  {
3      "count": 1,
4      "max": 1
5  }
```

11 Objets "place"

Une place sert à situer un élément dans le réseau et est spécifiée avec les attributs suivants :

"loop" (entier positif)

un indice de boucle

"element" (entier positif)

un indice d'élément

En voici un exemple :

```
1  Représentation d'un objet "place" :
2  {
3      "loop": 0,
4      "element": 3
5  }
```

12 Exemple de réseau complet

Voici pour exemple un réseau représentant en partie l'agglomération du *Grand Nancy*. Il est composé de 3 boucles, 4 ponts (donc 8 aiguillages - 4 entrants et 4 sortants), 8 gares, 3 capteurs et 2 dépôts. Et 4 capsules s'y déplacent (3 dans les boucles et 1 sur un pont).

```

1  {
2      "running": true,
3      "rate": 0,
4      "max_rate": 7,
5      "jerky": false,
6      "time": 29160,
7      "state": 3,
8      "name": "Mini network",
9      "view_box": {
10         "x": 0,
11         "y": 200,
12         "width": 600,
13         "height": 500
14     },
15     "margin_min": 2,
16     "max_speed": 30,
17     "pod_size": 2,
18     "place_number": 13,
19     "loops": [
20     {
21         "name": "Nancy",
22         "elements": [
23             {
24                 "type": "switch_in",
25                 "x": 434,
26                 "y": 676,
27                 "id_bridge": 0,
28                 "pods": []
29             },
30             {
31                 "type": "station",
32                 "name": "Gare",
33                 "x": 500,
34                 "y": 710,
35                 "pods": {"max": 4, "count": 0},
36                 "station_type": 0,
37                 "travelers": {"count": 0, "average_waiting_time": 0}
38             },
39             {
40                 "type": "station",
41                 "name": "Stanislas",
42                 "x": 580,
43                 "y": 630,
44                 "pods": {"max": 4, "count": 0},
45                 "station_type": 0,

```

```

46         "travelers": {"count": 0, "average_waiting_time": 0}
47     },
48     {
49         "type": "station",
50         "name": "Campus_ARTEM",
51         "x": 527,
52         "y": 555,
53         "pods": {"max": 4, "count": 0},
54         "station_type": 2,
55         "travelers": {"count": 0, "average_waiting_time": 0}
56     },
57     {
58         "type": "switch_out",
59         "x": 486, "y": 551,
60         "id_bridge": 1,
61         "pods": []
62     },
63     {
64         "type": "switch_out",
65         "x": 430,
66         "y": 590,
67         "id_bridge": 2,
68         "pods": []
69     },
70     {"type": "sensor", "x": 450, "y": 620}
71 ],
72 "sections": [
73     {"speed": 22.22, "path": {"type": "line"}},
74     {"speed": 22.22, "path": {"type": "line"}},
75     {"speed": 22.22, "path": {"type": "line"}},
76     {"speed": 22.22, "path": {"type": "line"}},
77     {"speed": 22.22, "path": {"type": "line"}},
78     {"speed": 22.22, "path": {"type": "line"}},
79     {"speed": 22.22, "path": {"type": "line"}}
80 ],
81 "pods": [
82     {
83         "position": 470,
84         "source": {"loop": 1, "element": 2},
85         "destination": {"loop": 1, "element": 1},
86         "travelers": {"count": 0, "max": 0}
87     }
88 ]
89 },
90 {
91     "name": "Villers-les-Nancy",

```

```

92     "elements": [
93         {
94             "type": "switch_in",
95             "x": 396,
96             "y": 456,
97             "id_bridge": 1,
98             "pods": []
99         },
100         {
101             "type": "station",
102             "name": "Velodrome",
103             "x": 411,
104             "y": 400,
105             "pods": {"max": 5, "count": 0},
106             "station_type": 0,
107             "travelers": {"count": 0, "average_waiting_time": 0}
108         },
109         {
110             "type": "station",
111             "name": "Telecom Nancy",
112             "x": 371,
113             "y": 315,
114             "pods": {"max": 4, "count": 0},
115             "station_type": 2,
116             "travelers": {"count": 0, "average_waiting_time": 0}
117         },
118         {
119             "type": "station",
120             "name": "Eglise",
121             "x": 222,
122             "y": 322,
123             "pods": {"max": 3, "count": 0},
124             "station_type": 1,
125             "travelers": {"count": 0, "average_waiting_time": 0}
126         },
127         {
128             "type": "shed",
129             "name": "Jean-paul",
130             "x": 189,
131             "y": 400,
132             "pods": {"max": 50, "count": 50}
133         },
134         {
135             "type": "switch_out",
136             "x": 215,
137             "y": 471,

```

```

138         "id_bridge": 3,
139         "pods": []
140     },
141     {"type": "sensor", "x": 280, "y": 480}
142 ],
143 "sections": [
144     {"speed": 22.22, "path": {"type": "line"}},
145     {"speed": 22.22, "path": {"type": "line"}},
146     {"speed": 22.22, "path": {"type": "line"}},
147     {"speed": 22.22, "path": {"type": "line"}},
148     {"speed": 22.22, "path": {"type": "line"}},
149     {"speed": 22.22, "path": {"type": "line"}},
150     {"speed": 22.22, "path": {"type": "line"}}
151 ],
152 "pods": [
153     {
154         "position": 630,
155         "source": {"loop": 1, "element": 2},
156         "destination": {"loop": 1, "element": 1},
157         "travelers": {"count": 0, "max": 0}
158     }
159 ],
160 },
161 {
162     "name": "Laxou",
163     "elements": [
164         {
165             "type": "switch_in",
166             "x": 222,
167             "y": 526,
168             "id_bridge": 2,
169             "pods": []
170         },
171         {
172             "type": "switch_in",
173             "x": 78,
174             "y": 526,
175             "id_bridge": 3,
176             "pods": []
177         },
178         {
179             "type": "station",
180             "name":
181             "Auchan",
182             "x": 7,
183             "y": 650,

```



```

184         "pods": {"max": 3, "count": 0},
185         "station_type": 2,
186         "travelers": {"count": 0, "average_waiting_time": 0}
187     },
188     {
189         "type": "station",
190         "name": "Sapiniere",
191         "x": 101,
192         "y": 785,
193         "pods": {"max": 3, "count": 0},
194         "station_type": 1,
195         "travelers": {"count": 0, "average_waiting_time": 0}
196     },
197     {
198         "type": "shed",
199         "name": "Marie-Jeanne",
200         "x": 175,
201         "y": 791,
202         "pods": {"max": 100, "count": 100}
203     },
204     {
205         "type": "switch_out",
206         "x": 260,
207         "y": 742,
208         "id_bridge": 0,
209         "pods": []
210     },
211     {"type": "sensor", "x": 280, "y": 650}
212 ],
213 "sections": [
214     {"speed": 22.22, "path": {"type": "line"}},
215     {"speed": 22.22, "path": {"type": "line"}},
216     {"speed": 22.22, "path": {"type": "line"}},
217     {"speed": 22.22, "path": {"type": "line"}},
218     {"speed": 22.22, "path": {"type": "line"}},
219     {"speed": 22.22, "path": {"type": "line"}},
220     {"speed": 22.22, "path": {"type": "line"}}
221 ],
222 "pods": [
223     {
224         "position": 840,
225         "source": {"loop": 1, "element": 2},
226         "destination": {"loop": 1, "element": 1},
227         "travelers": {"count": 0, "max": 0}
228     }
229 ]

```

```

230     }
231 ],
232 "bridges": [
233     {
234         "name": "bridge 0",
235         "section": {"speed": 22.22, "path": {"type": "line"}},
236         "pods": [
237             {
238                 "position": 100,
239                 "source": {"loop": 1, "element": 2},
240                 "destination": {"loop": 1, "element": 1},
241                 "travelers": {"count": 0, "max": 0}
242             }
243         ]
244     },
245     {
246         "name": "bridge 1",
247         "section": {"speed": 22.22, "path": {"type": "line"}},
248         "pods": []
249     },
250     {
251         "name": "bridge 2",
252         "section": {"speed": 22.22, "path": {"type": "line"}},
253         "pods": []
254     },
255     {
256         "name": "bridge 3",
257         "section": {"speed": 22.22, "path": {"type": "line"}},
258         "pods": []
259     }
260 ]
261 }

```